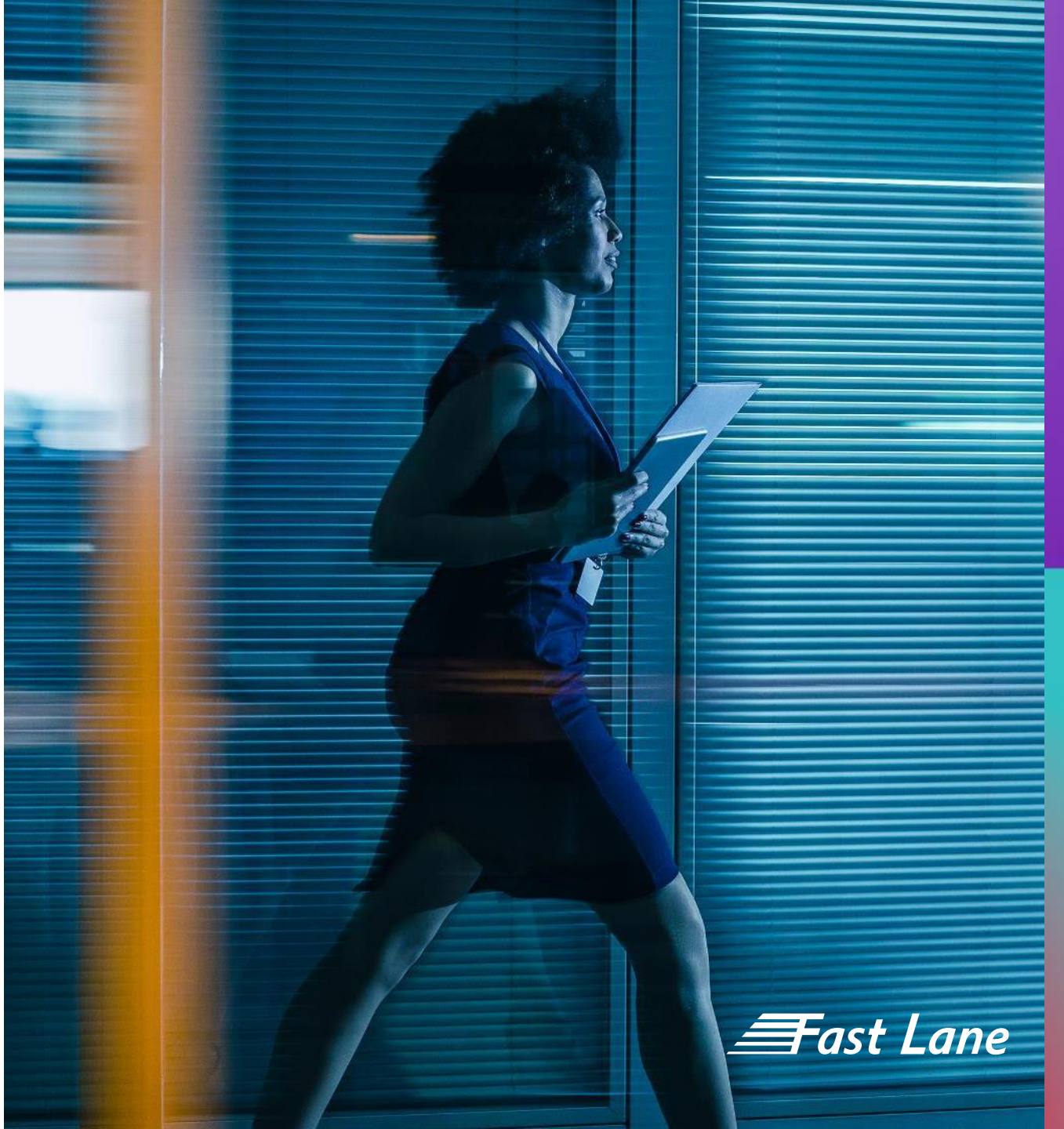
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Integrations IT Solution e. U.

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE — ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT — ★★★

SCAN TO CONNECT
integrations.at

MODULE 3

Copilot Studio Basics

A working, grounded agent in Teams, built without code

Automate Business Processes with Copilot Studio and Copilot Cowork



01

Standard Agents: Create & Configure

Identity, instructions, model, orchestration and the test panel

02

Knowledge Sources & Suggested Prompts

Source types, citations and ungrounded answers

03

Topics, Entities & Variables

Nodes, entities, slot filling and variable scopes

04

Power Fx: Formulas in Topics

Where formulas run and the functions makers use most

05

Adaptive Cards: Collect Structured Input

Card JSON, schema versions and outputs as variables

06

Publishing to Teams & Copilot

Channels, authentication and who can use the agent

Copilot Studio Basics

A working, grounded agent in Teams, built without code

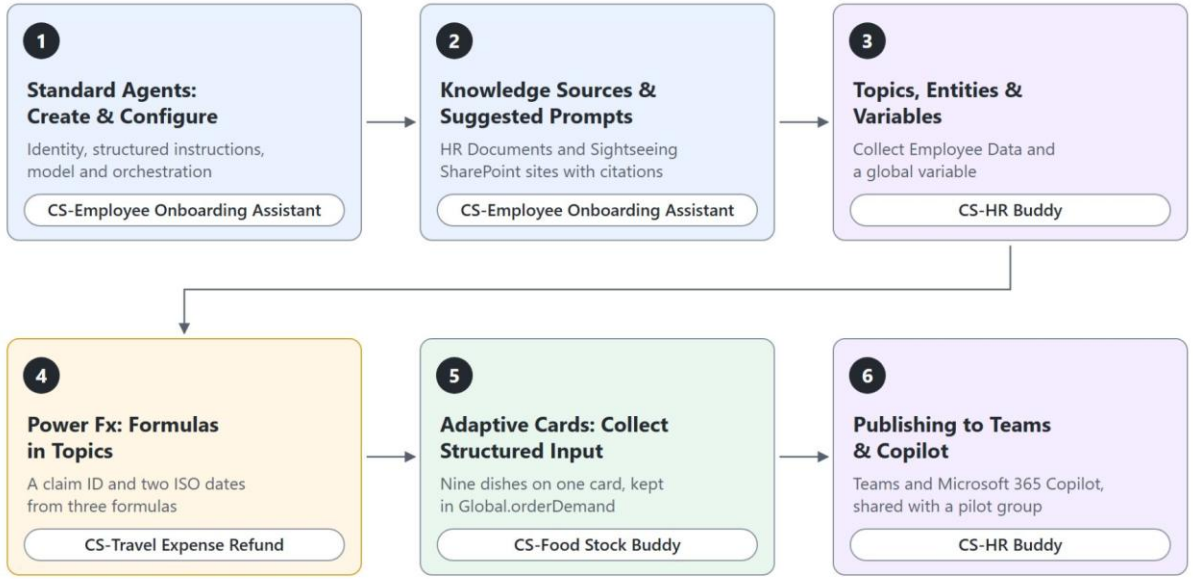
- Create a **Standard agent** and set its instructions, model and orchestration
- Ground it in **knowledge** and shape its conversations with topics, Power Fx and Adaptive Cards
- **Publish** it to Teams and Microsoft 365 Copilot



Overview

Six topics, four agents rebuilt in the demos

- Create **CS-Employee Onboarding Assistant** and ground it in two SharePoint sites
- Build **CS-HR Buddy** topics that identify an employee once
- Turn a travel claim into values ready to save with **Power Fx**
- Collect a canteen demand on one **Adaptive Card**
- Publish CS-HR Buddy to **Teams and Microsoft 365 Copilot**



01

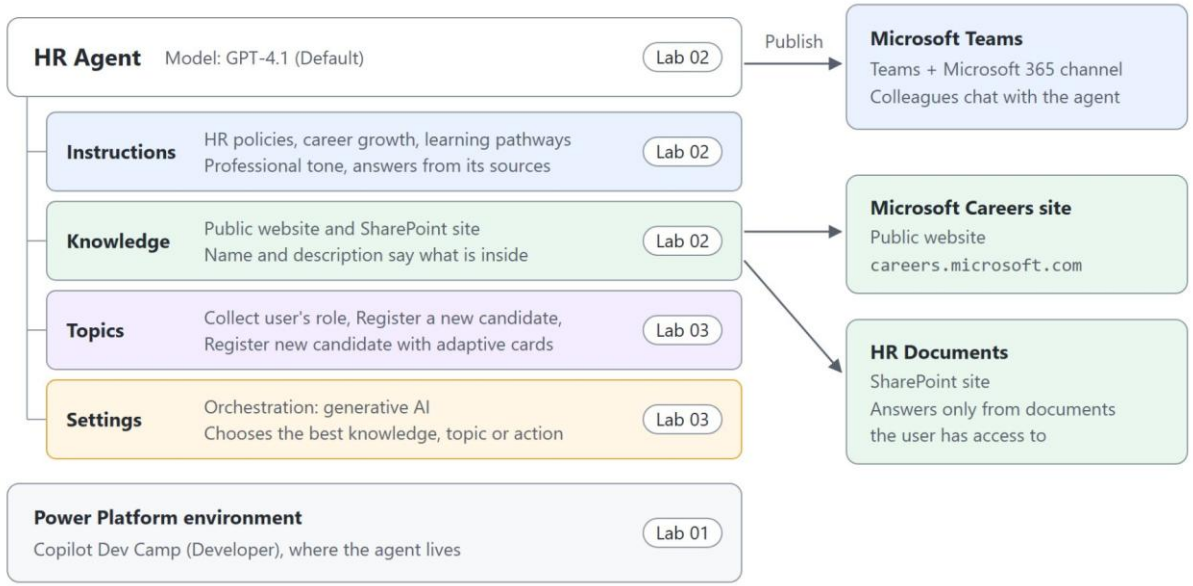
Standard Agents: Create & Configure

Identity, instructions, model, orchestration and the test panel

Parts of an agent

The HR Agent built in labs 01 to 03

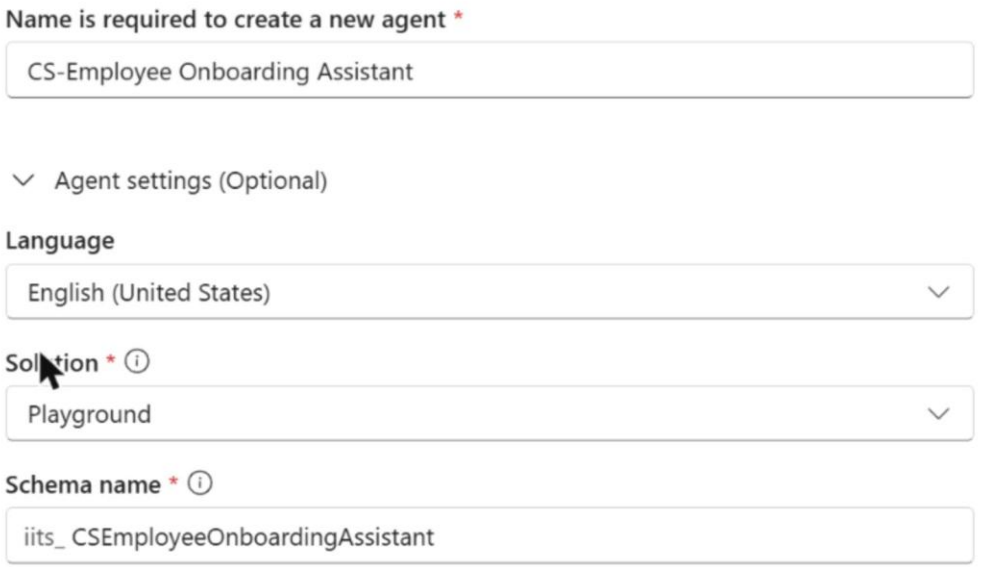
- Every agent lives in a Power Platform **environment** with its Dataverse database and solutions
- Name, description and **instructions** set its identity and tone
- **Knowledge** grounds its answers in websites, SharePoint, files or Dataverse
- **Topics** carry the conversations you want to control precisely



Creating a Standard agent

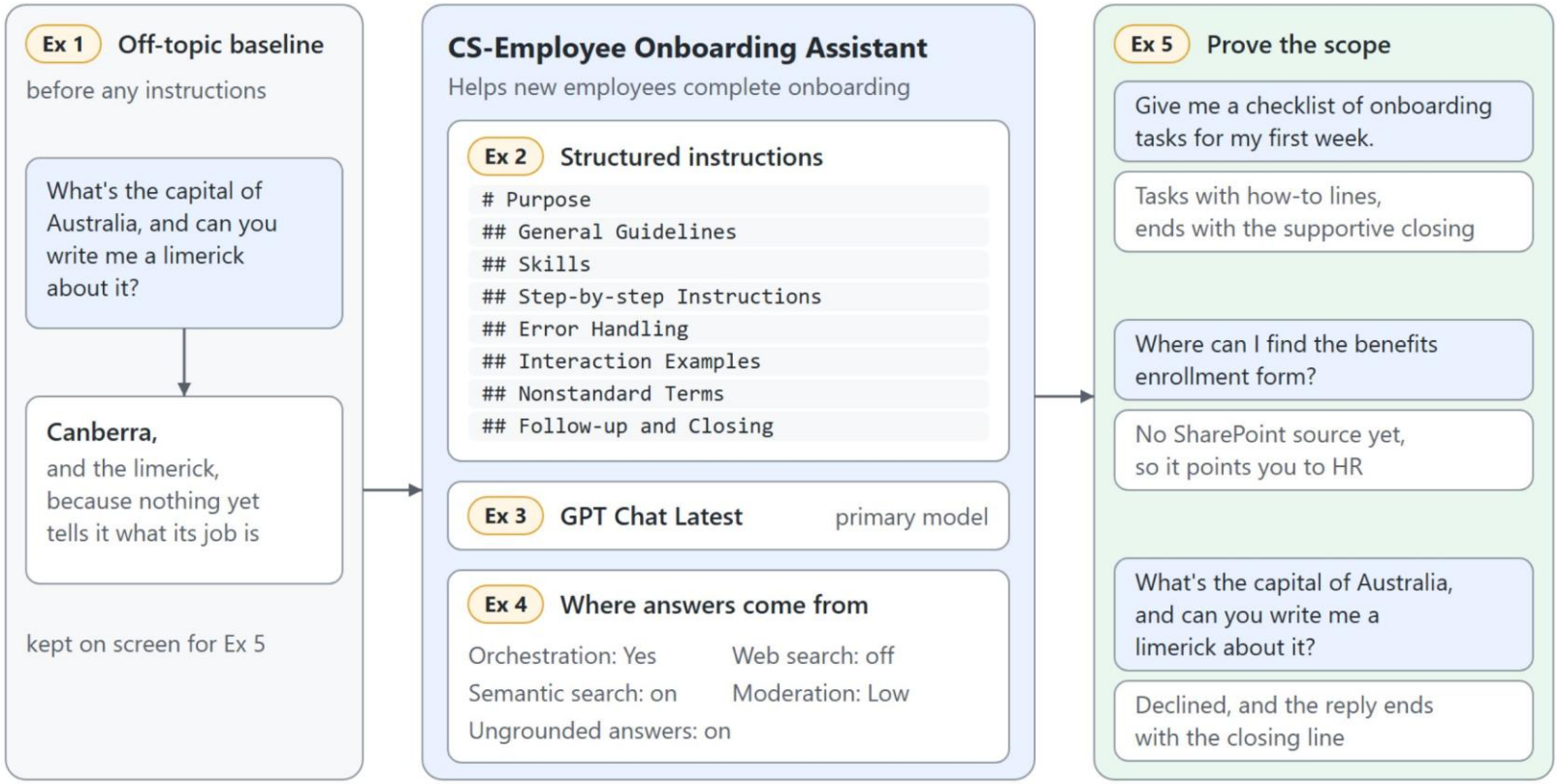
Create blank agent for CS-Employee Onboarding Assistant

- Three starts: **describe** the agent, **Create blank agent**, or an agent template
- A blank agent needs only a **name**
- **Agent settings** show language, solution and schema name; the demo keeps the defaults
- The agent stays in the **environment** selected before you create it



Instructions, model and answer settings

The same off-topic question before and after



Structured **instructions**, GPT Chat Latest and deliberate answer settings keep CS-Employee Onboarding Assistant on onboarding questions

02

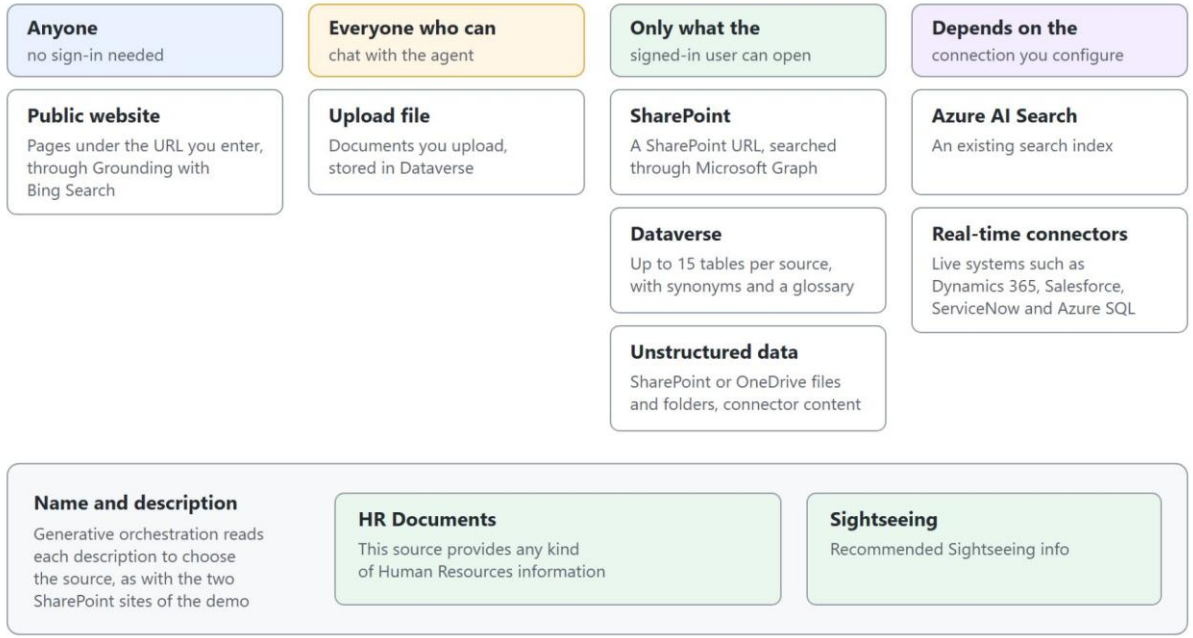
Knowledge Sources & Suggested Prompts

Source types, citations and ungrounded answers

Knowledge source types

What the agent searches and who can see the content

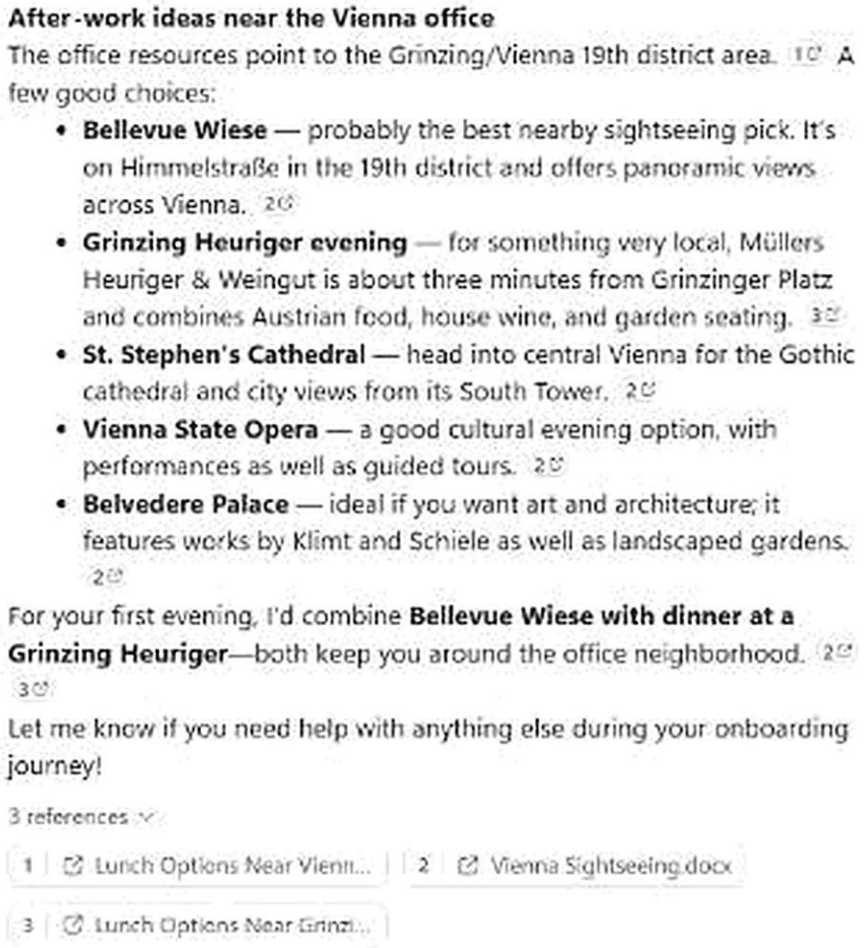
- **Add knowledge** offers websites, files, SharePoint, Dataverse, Azure AI Search and connectors
- SharePoint and Dataverse answer within the **signed-in user's** permissions
- An **uploaded file** can answer everyone who chats with the agent
- Each source gets a **name and description** the orchestrator chooses by



Answers with citations

The sightseeing question answered from SharePoint

- The question: what to see near the office after work
- The **Sightseeing** source and one instruction section steer it to the Sightseeing site
- Five suggestions, each with a numbered **citation**
- **3 references** under the answer link to the documents



After-work ideas near the Vienna office

The office resources point to the Grinzing/Vienna 19th district area. 1 A few good choices:

- **Bellevue Wiese** — probably the best nearby sightseeing pick. It's on Himmelstraße in the 19th district and offers panoramic views across Vienna. 2
- **Grinzing Heuriger evening** — for something very local, Müllers Heuriger & Weingut is about three minutes from Grinzinger Platz and combines Austrian food, house wine, and garden seating. 3
- **St. Stephen's Cathedral** — head into central Vienna for the Gothic cathedral and city views from its South Tower. 2
- **Vienna State Opera** — a good cultural evening option, with performances as well as guided tours. 2
- **Belvedere Palace** — ideal if you want art and architecture; it features works by Klimt and Schiele as well as landscaped gardens. 2

For your first evening, I'd combine **Bellevue Wiese with dinner at a Grinzing Heuriger**—both keep you around the office neighborhood. 2 3

Let me know if you need help with anything else during your onboarding journey!

3 references

1 Lunch Options Near Vienn... | 2 Vienna Sightseeing.docx

3 Lunch Options Near Grinzi...

Grounded and ungrounded answers

What Allow ungrounded responses changes

An ungrounded answer sounds just as right as a grounded one, so open the **citation**: the cited file proves grounding



03

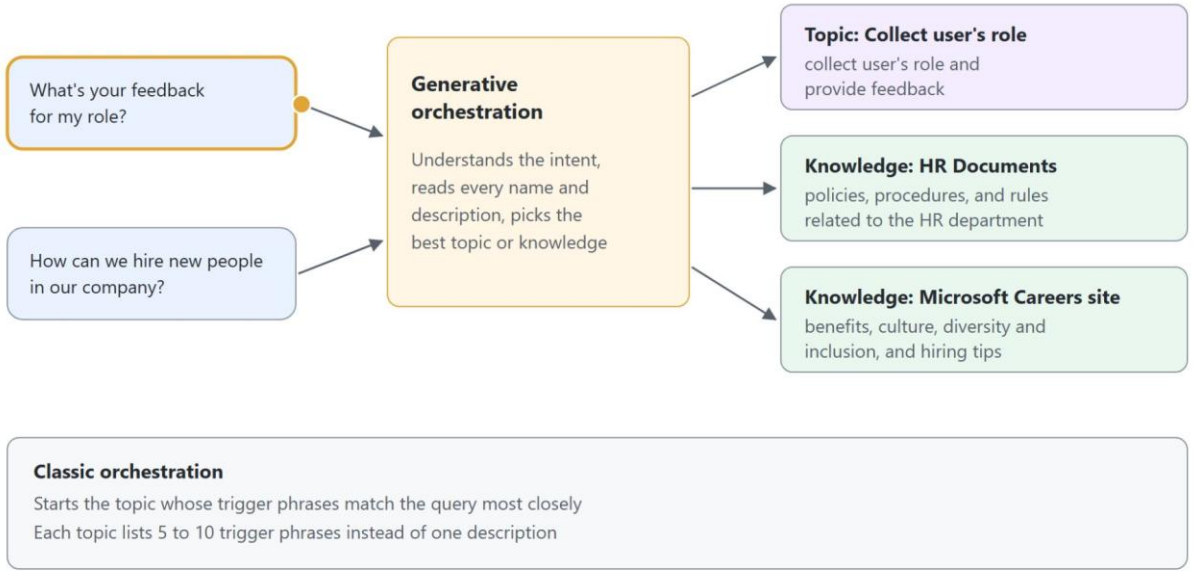
Topics, Entities & Variables

Nodes, entities, slot filling and variable scopes

Topics and generative answers

How orchestration picks a topic or knowledge for each prompt

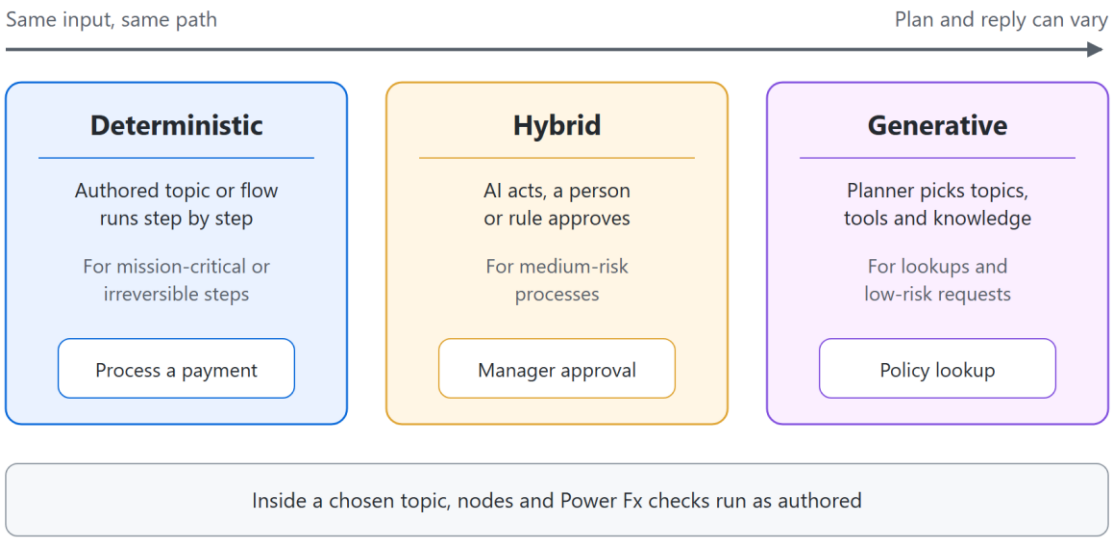
- A **topic** runs a fixed path of nodes: questions, messages, conditions, redirects
- **Knowledge** with generative answers covers open-ended questions over your content
- Generative orchestration picks a topic by its **description**
- Classic orchestration starts the topic whose **trigger phrases** match most closely



Deterministic and non-deterministic orchestration

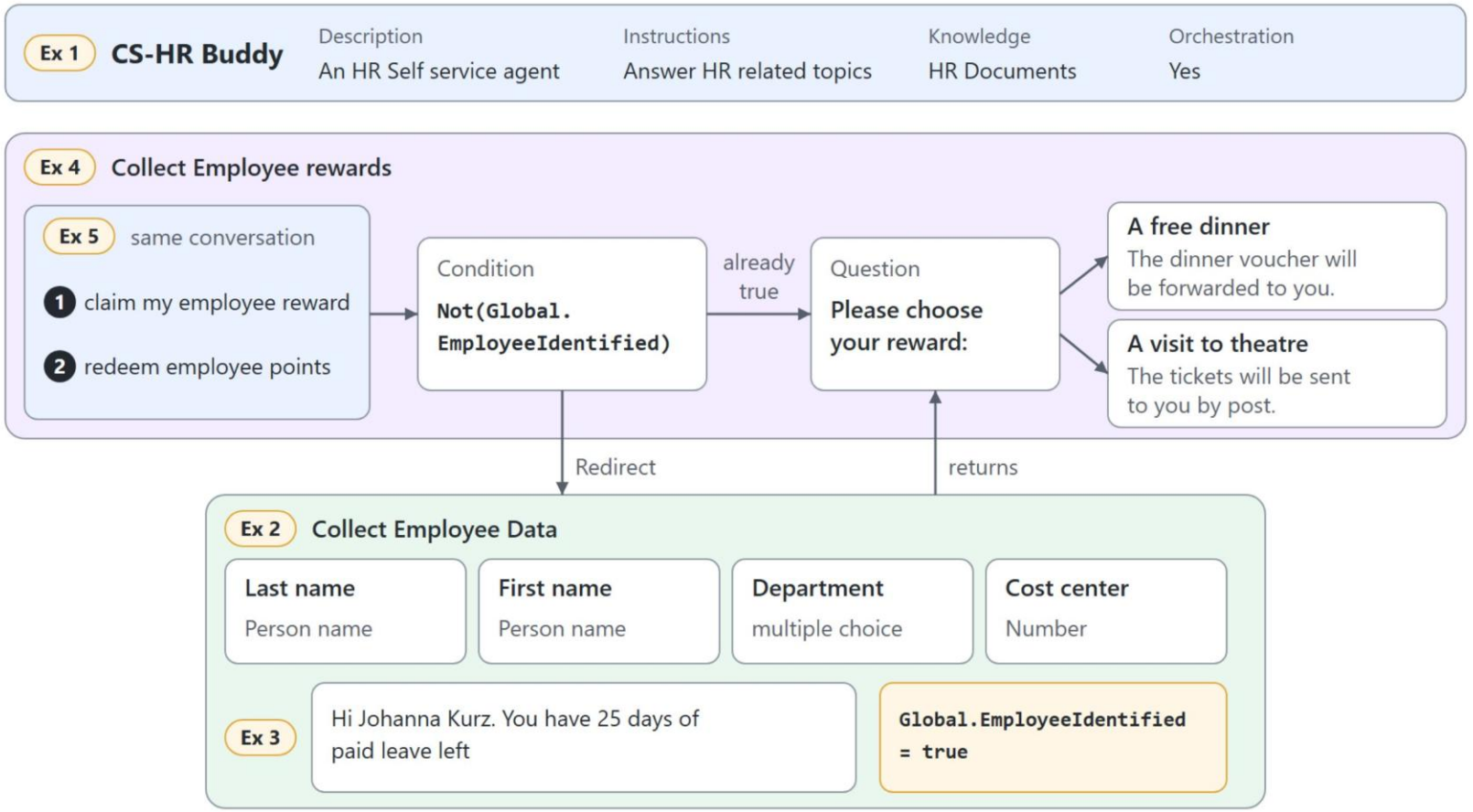
Which parts of an agent run the same way every time

- Classic orchestration routes **deterministically** by trigger phrases
- Generative orchestration **plans** each turn from descriptions
- Its reply to the same question can **differ** between runs
- A chosen topic runs its **nodes and checks** as authored



Sharing state between topics

CS-HR Buddy identifies an employee once

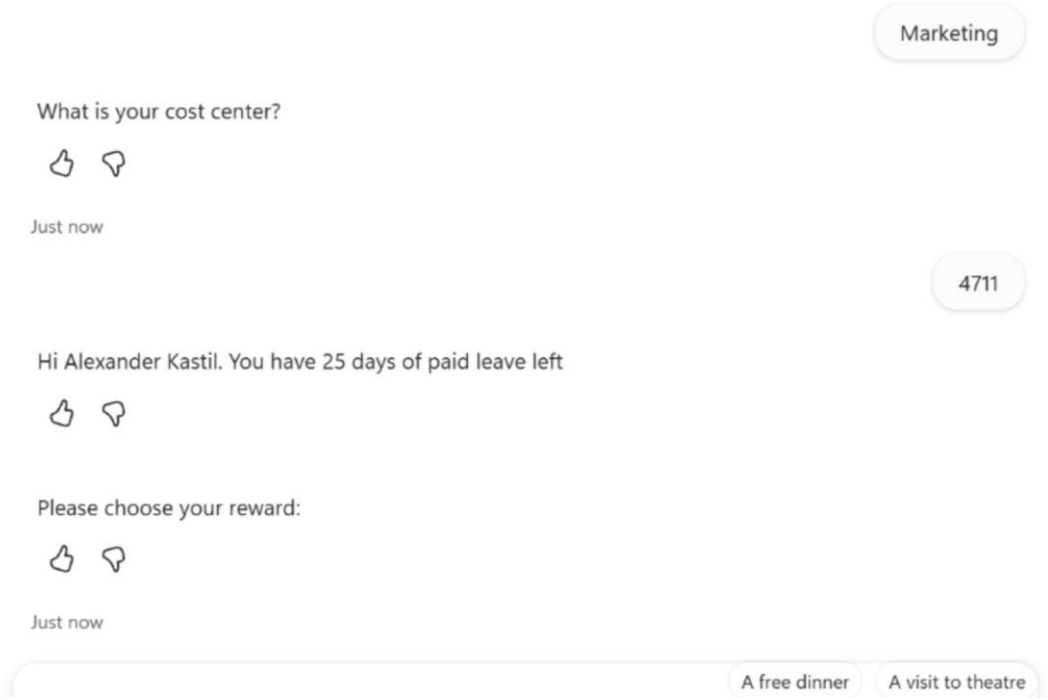


Collect Employee Data sets **Global.EmployeeIdentified**, so the rewards topic detours through it only until the employee is known

The rewards detour in the test chat

The first reward request runs the employee questions

- A reward request from an unknown employee runs **Collect Employee Data** first
- The cost center is recognized as a **Number**
- The paid leave message confirms the employee
- The redirect returns to **Please choose your reward:**



04

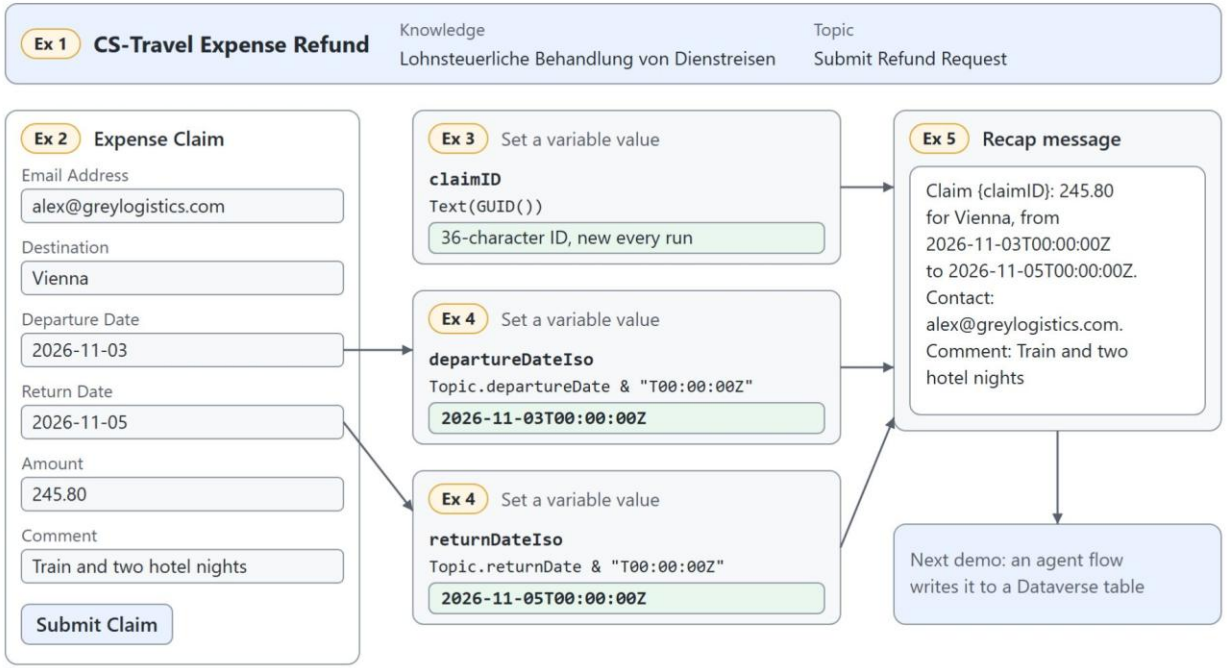
Power Fx: Formulas in Topics

Where formulas run and the functions makers use most

Power Fx in a topic

Three formulas turn the expense card into a claim ready to save

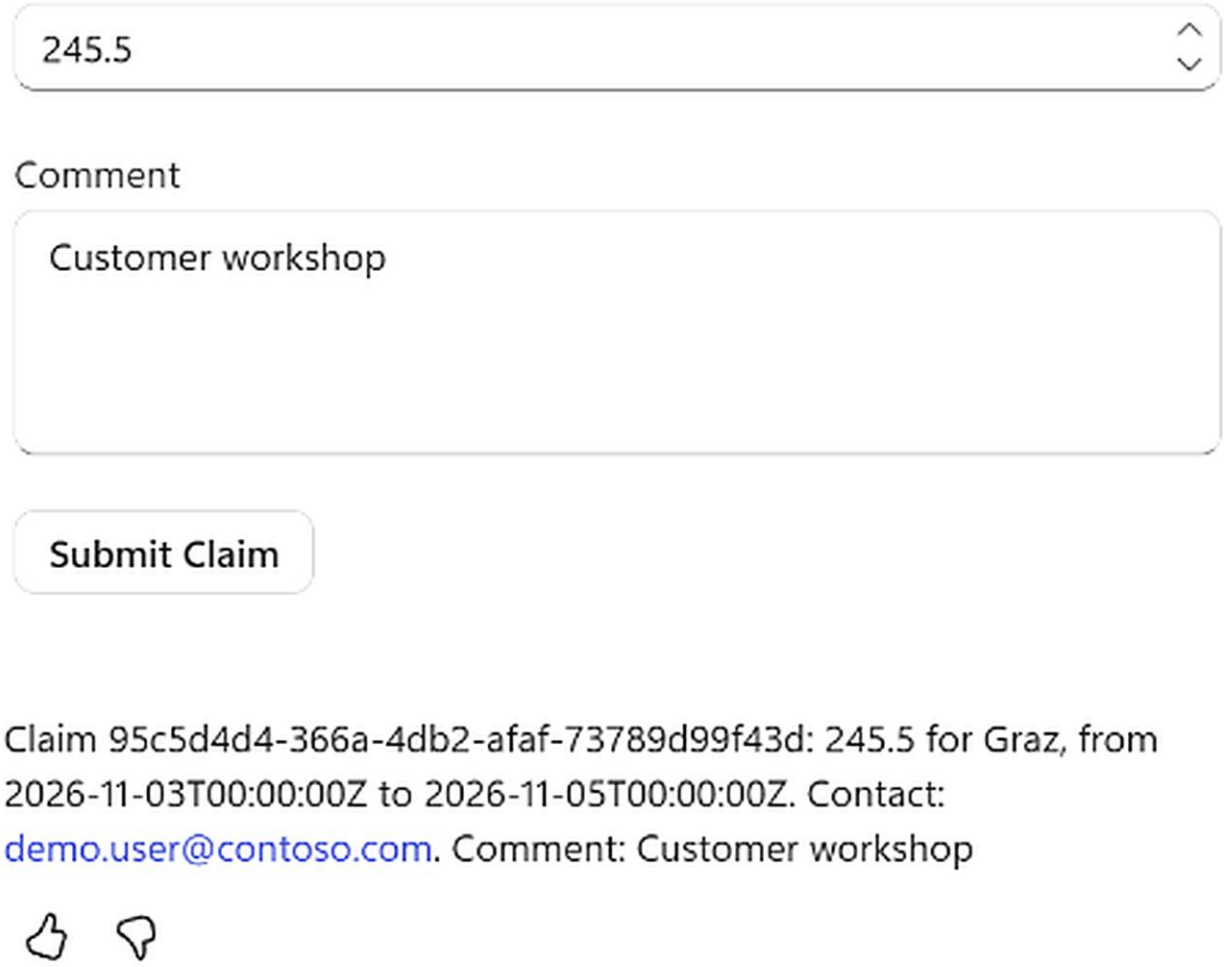
- Formulas run wherever a node takes a value: **Set variable value**, Condition, Question
- `Text(GUID())` gives every claim its own **claimID**
- `& "T00:00:00Z"` turns each card date into an **ISO timestamp**
- Name variables with their **scope prefix**, separate arguments with commas



The claim recap

The values an agent flow receives next

- **Submit Claim** fills one topic variable per card input
- The recap starts with the **claimID**, a 36-character GUID
- Both dates arrive as **ISO timestamps** ending in T00:00:00Z
- A second test run gets a **different claimID**



05

Adaptive Cards: Collect Structured Input

Card JSON, schema versions and outputs as variables

Canteen demand on one card

CS-Food Stock Buddy collects nine dishes in one submit

One **Adaptive Card** replaces nine questions: the canteen manager types a number per dish and submits once

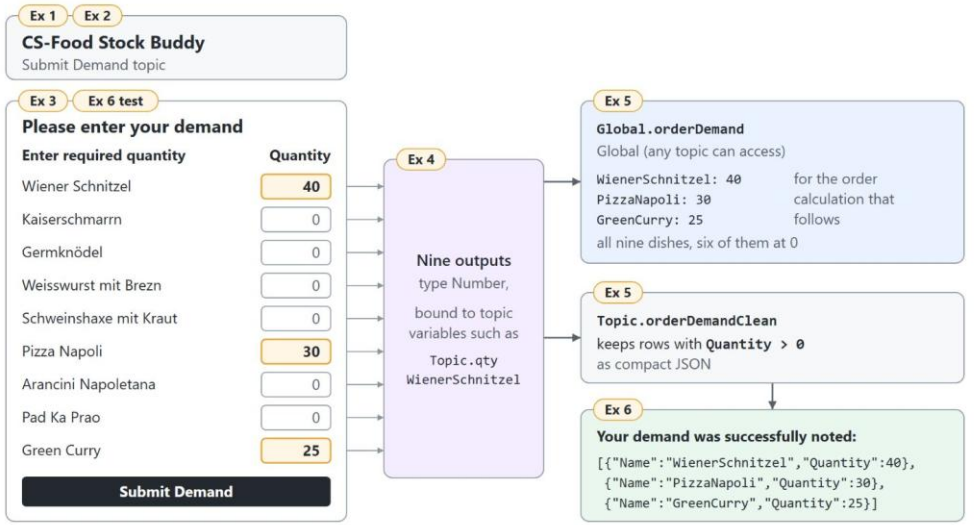


From card inputs to variables

Input id, binding and type must agree

- Each input **id** becomes a typed output bound to a topic variable
- `Global.orderDemand` keeps all nine numbers for the order calculation

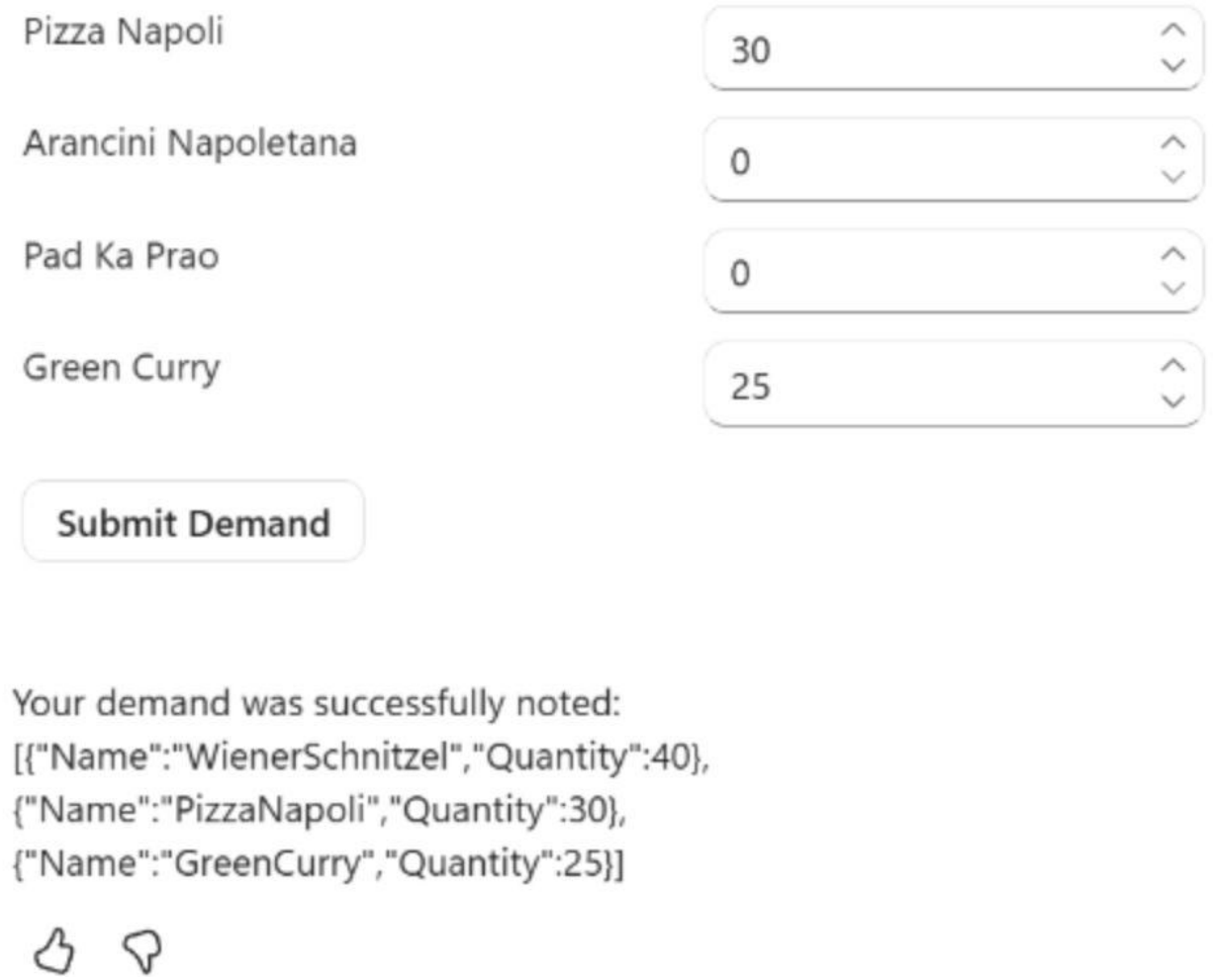
```
output:
  binding:
    qty.WienerSchnitzel: Topic.qtyWienerSchnitzel
    qty.PizzaNapoli: Topic.qtyPizzaNapoli
    qty.GreenCurry: Topic.qtyGreenCurry
outputType:
  properties:
    qty.WienerSchnitzel: Number
```



Submitting the demand

40 schnitzel, 30 pizza and 25 curry in one submit

- Each dish row has a **number field** that starts at 0
- Pizza Napoli 30 and Green Curry 25 typed, the other rows left at 0
- **Submit Demand** sends every row in one step
- The confirmation lists only the **three dishes** above 0



06

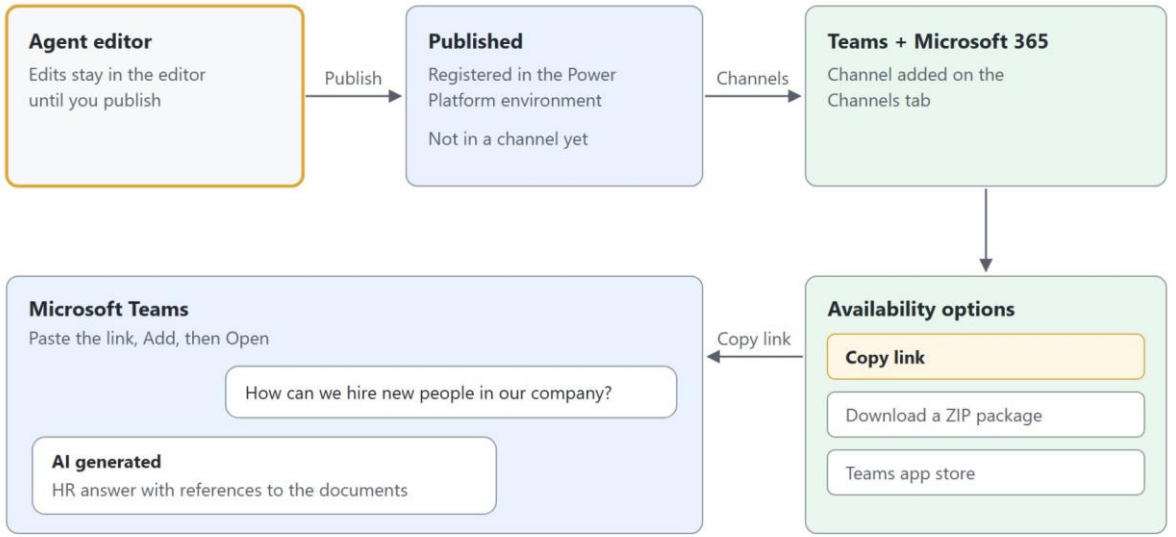
Publishing to Teams & Copilot

Channels, authentication and who can use the agent

Publishing to channels

From the agent editor to a chat in Microsoft Teams

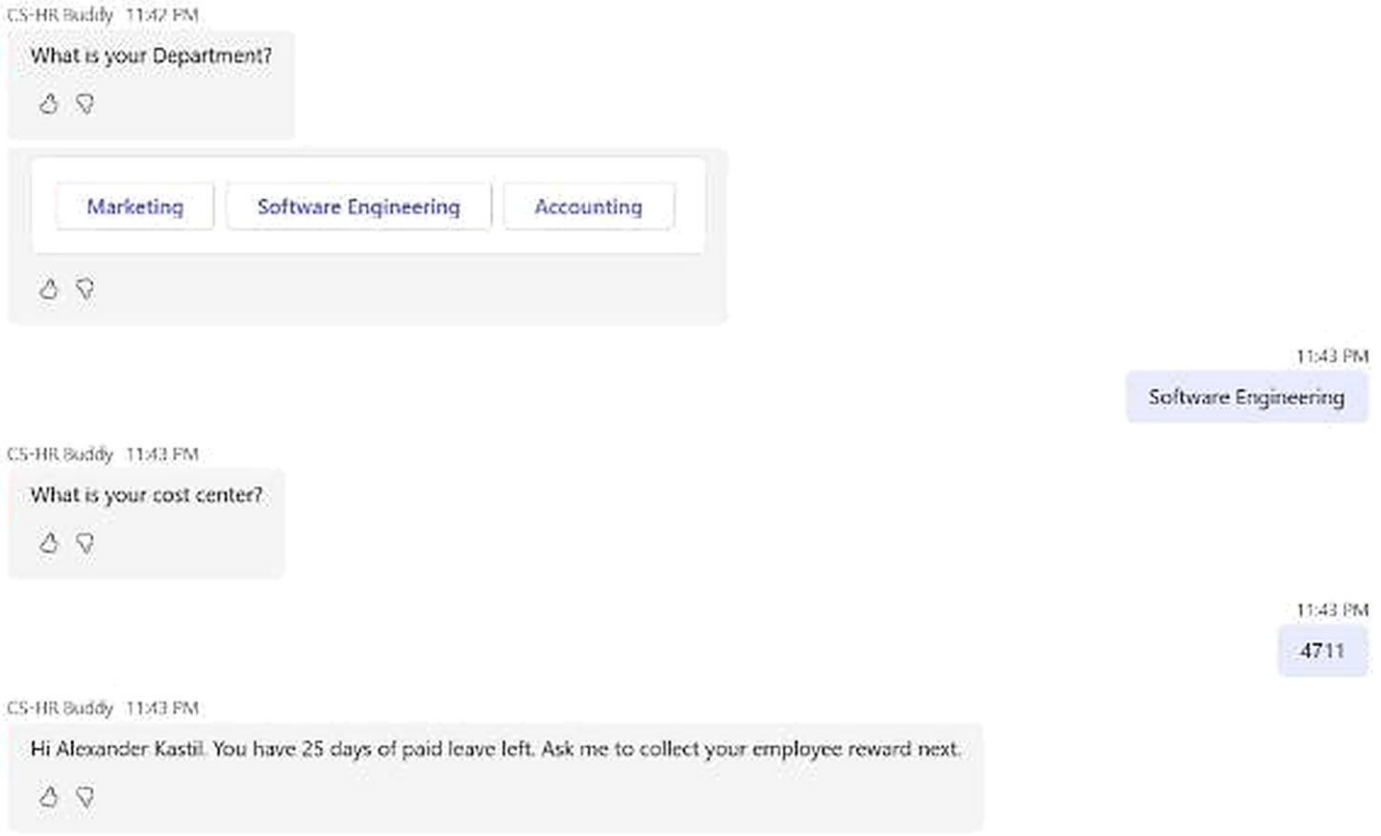
- **Publish** copies the draft to every channel the agent is connected to
- The **Microsoft 365 and Microsoft Teams** channel is added after the first publish
- **Availability options** decide who finds the agent: link, shared users, the org, or a .zip
- **Authenticate with Microsoft** opens Teams, Microsoft 365 Copilot and SharePoint



CS-HR Buddy in Teams

The republished message after start over

- The published agent runs the same **topics** in a Teams chat
- The department options appear as **buttons**
- After **Publish** and `start over`, the message ends with the new sentence
- Start over also clears **Global.EmployeeIdentified**



Draft and published version

Who sees which version, and who may chat

You edit a draft, colleagues chat with the **published** version, and sharing decides who may chat at all



Takeaways

What the tools module builds on

- Write **instructions** that state the job, the limits and how a conversation ends
- Give every knowledge source and topic a **description** the orchestrator can choose by
- Trust an answer once its **citation** opens the source
- Keep fixed paths in **topics**, logic in **Power Fx** and multi-field input on **Adaptive Cards**
- Users chat with the **published** version, and only once you share it

